REAL-TIME COMPUTER VISION

YOLOv8n Model Optimization

Benchmark Report: FP32, FP16 and Input Resolution

Project: Real-Time Object Detection and Tracking

Model: YOLOv8n (yolov8n.pt)

Benchmark device: NVIDIA Tesla T4 GPU on Colab

Measurement: 100 timed runs after 20 warm-up runs

Prepared from the measured benchmark files supplied for this submission. Results are reported as observed; no performance values have been fabricated.

1. Executive Summary

This report evaluates three YOLOv8n inference configurations using the same model checkpoint: a baseline FP32 run at 640 pixels, an FP16 run at 640 pixels, and an FP16 run at 512 pixels. The benchmark used a Tesla T4 GPU, 20 warm-up iterations, and 100 timed runs per configuration.

The measured baseline achieved **7.547 ms** mean latency and **132.51** FPS derived from mean latency. In this run, neither tested FP16 configuration improved speed: FP16 at 640 was 11.9% slower by mean latency, and FP16 at 512 was 14.6% slower.

The central engineering result is therefore not a speedup claim. It is a measured comparison showing that enabling half precision and reducing image size did not improve latency in this particular setup. The result is useful because optimization choices must be validated on the target hardware and end-to-end workload rather than assumed to be faster.

Key findings

Baseline	7.547 ms · 132.51 FPS
FP16 at 640	8.443 ms · 118.43 FPS; slower than baseline
FP16 at 512	8.651 ms · 115.59 FPS; slower than baseline
Model file size	6.246 MB in all three reported configurations
Accuracy evidence	COCO8 smoke-test metrics only; not enough to establish production accuracy

Recommendation: retain FP32 at 640 as the current benchmark winner until a controlled re-run or larger validation set demonstrates a better configuration.

2. Objective and Computational Bottlenecks

The project uses a pretrained YOLOv8n detector as part of a real-time detection-and-tracking pipeline. A typical frame passes through image capture, preprocessing, model inference, post-processing (including boxes and confidence scores), tracker association, trajectory updates/drawing, and display. The benchmark provided here measures model inference configurations, not the complete webcam-to-display pipeline.

Likely cost centers to investigate

Model inference: convolutional feature extraction and detection-head operations are the main neural-network computation. Input dimensions affect the amount of image data processed.

Preprocessing and post-processing: resizing, tensor conversion, non-maximum suppression, and conversion of model outputs can add latency beyond raw neural-network execution.

Tracking and visualization: persistent ID association, trajectory storage, drawing, and window display add per-frame work. These costs are not isolated by the supplied benchmark.

Measurement overhead: GPU kernels can execute asynchronously. Reliable GPU timing requires appropriate synchronization and consistent timing boundaries; warm-up iterations help reduce one-time initialization effects.

Important scope limitation

The uploaded environment reports **video: null**. Consequently, the supplied results do not demonstrate end-to-end webcam tracking FPS, ID stability, or trajectory rendering speed. They are model benchmark results under the reported configuration.

3. Experimental Setup

Item	Recorded value
GPU	Tesla T4
Platform	Linux-6.6.122+-x86_64-with-glibc2.39
Python	3.13.15
PyTorch	2.11.0+cu130
Ultralytics	8.4.182
OpenCV	4.14.0
CUDA	13.0
Model	yolov8n.pt
Warm-up / timed runs	20 / 100 per configuration
Validation dataset	COCO8 (small smoke-test dataset)

4. Optimization Techniques Tested

4.1 FP32 baseline at 640 × 640

FP32 stores floating-point values using 32 bits. This configuration is the comparison baseline: yolov8n.pt, image size 640, and half precision disabled. It provides the reference latency and detection metrics for evaluating the other settings.

4.2 FP16 inference at 640 × 640

FP16 uses 16-bit floating-point values where supported. It can reduce activation memory and improve throughput on compatible hardware, but actual gains depend on the GPU, kernels, precision handling, workload size, and runtime overhead. In the measured results, the FP16 request at 640 did not make inference faster: mean latency increased from 7.547 ms to 8.443 ms. The recorded model file size remained 6.246 MB because requesting half precision at inference does not necessarily rewrite the stored checkpoint as a half-precision file.

4.3 FP16 inference at 512 × 512

Reducing input dimensions from 640 to 512 decreases the number of input pixels from 409,600 to 262,144, a reduction of 36%. This can reduce some model computation, but latency is not determined by pixel count alone. Fixed overhead, kernel selection, memory movement, preprocessing, and measurement variation can offset expected savings. In this run, the 512 configuration had the highest mean latency (8.651 ms), so a speed improvement was not observed.

4.4 Techniques not implemented in these results

Pruning: no structured or unstructured pruning experiment is present in the supplied result files. It must not be reported as implemented.

INT8 quantization: no INT8 conversion or calibration experiment is present. FP16 is reduced-precision floating point, not INT8 quantization.

ONNX / TensorRT: no exported-runtime benchmark is included. These are recommended next experiments, not completed work.

Multi-threading / batch processing: the supplied results do not compare these techniques. Batch processing may improve throughput but can increase per-frame latency and is not always appropriate for interactive video.

5. Benchmark Results

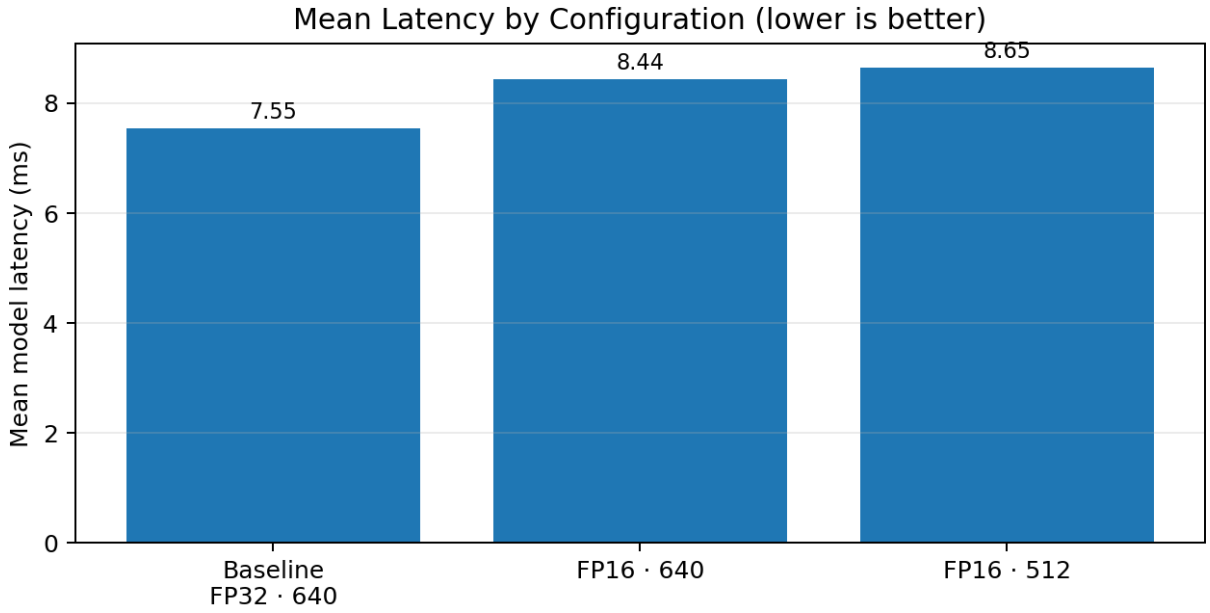
The values below are taken directly from benchmark_results.csv / benchmark_results.json. FPS is calculated from mean latency (1,000 ÷ mean milliseconds); it is a throughput proxy for the measured model call, not measured end-to-end application FPS.

Configuration	Input	Precision	Mean ms	Median ms	P95 ms	FPS	mAP50	mAP50–95
fp32_640	640	FP32	7.547	7.105	9.664	132.51	0.8957	0.6299
fp16_640	640	FP16	8.443	8.088	10.469	118.43	0.8957	0.6278
fp16_512	512	FP16	8.651	8.249	10.643	115.59	0.8965	0.7046

Additional baseline metrics: precision 0.6210 and recall 0.8333. FP16 at 640: precision 0.6253 and recall 0.8333. FP16 at 512: precision 0.6805 and recall 0.7500. These metrics are specific to the small COCO8 validation run.

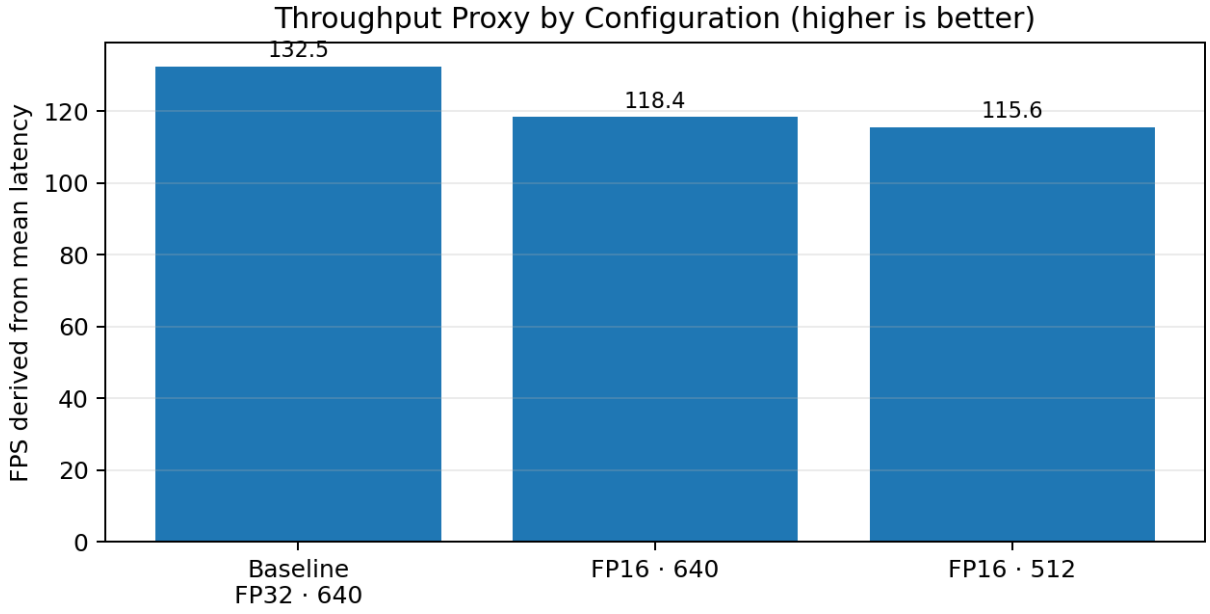
6. Performance Graphs

6.1 Mean inference latency



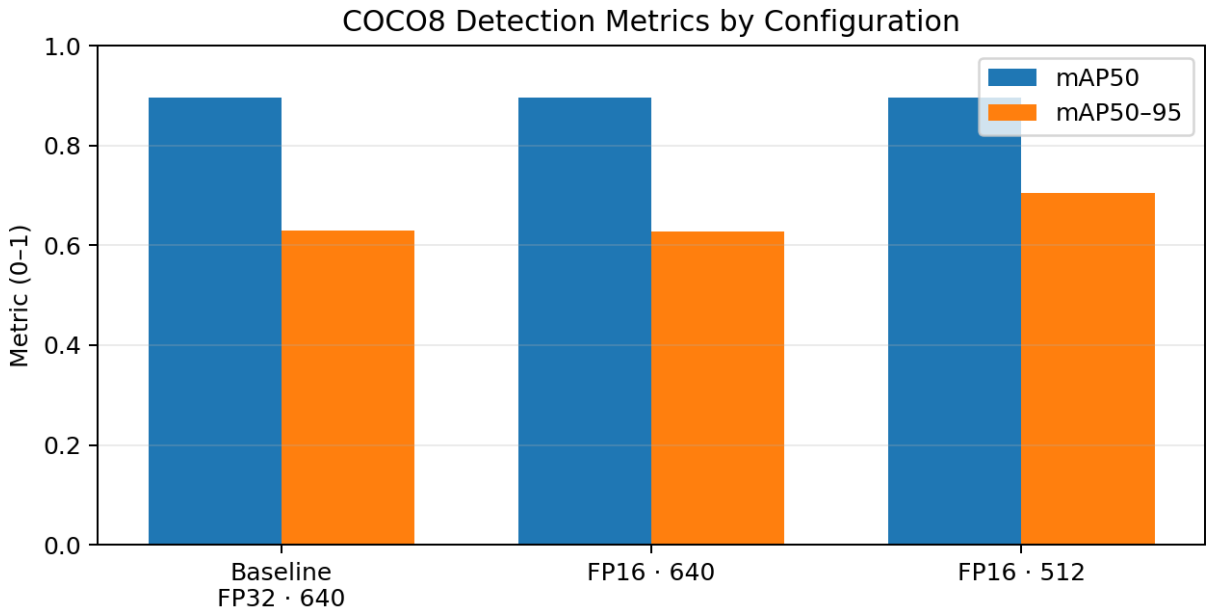
Lower is better. Baseline mean latency was 7.547 ms. FP16 at 640 was +11.9% versus baseline; FP16 at 512 was +14.6%.

6.2 FPS derived from mean latency



Higher is better. The baseline produced the highest derived FPS. These values are the reciprocal of the mean latency, not an independent end-to-end video FPS measurement.

6.3 Detection quality metrics



The FP16 512 run reports higher mAP50–95 than the baseline, while recall falls from 0.8333 to 0.7500. Given the small COCO8 dataset and absence of a custom test set, this difference should be treated as a preliminary observation rather than proof that lower resolution improves accuracy.

7. Interpretation and Trade-offs

Comparison	Mean latency change	FPS change	mAP50 change	mAP50–95 change
FP16 · 640 vs baseline	+11.9%	-10.6%	-0.0000	-0.0021
FP16 · 512 vs baseline	+14.6%	-12.8%	+0.0008	+0.0747

The tested configurations did not meet the intended goal of reducing measured latency. FP16 at 640 retained almost the same mAP50 and recall as baseline, but was slower. FP16 at 512 showed a higher mAP50–95 value in this run, but also lower recall and slower latency. Because COCO8 is small, small metric differences can be unstable.

Why might FP16 be slower here?

Potential explanations to test—not confirmed causes—include small-model overheads, runtime/kernel selection, precision conversion or fallback operations, timing variation, and benchmark scope. The results do not contain operator-level profiling that would identify the cause. A profiler trace and repeated runs on a larger dataset would be needed to diagnose it.

What can be concluded?

For this exact Tesla T4 / software / benchmark setup, FP32 at 640 was the fastest of the three tested configurations. This is not a universal claim that FP32 is faster than FP16; it is a result of this experiment. Do not generalize it to other GPUs, drivers, model sizes, or video pipelines.

8. Code Changes and Reproducibility

The supplied benchmark package separates the baseline and optimized inference entry points and runs a controlled comparison. The important configuration difference is explicit rather than hidden in the code.

Representative inference calls

```
# Baseline: FP32 at 640
results = model.predict(
    source=frame, imgsz=640, device=0,
    half=False, verbose=False
)

# Experiment A: FP16 at 640
results = model.predict(
    source=frame, imgsz=640, device=0,
    half=True, verbose=False
)

# Experiment B: FP16 at 512
results = model.predict(
    source=frame, imgsz=512, device=0,
    half=True, verbose=False
)
```

For a tracking application, the corresponding model.track(...) call can use the same settings and persistent tracking state. However, these supplied measurements were generated with video set to null, so the report does not claim these timings include tracking, camera capture, trajectory drawing, or display.

Benchmark procedure

1. Load the same yolov8n.pt checkpoint for each configuration. 2. Apply 20 warm-up runs. 3. Record 100 timed runs per configuration. 4. Record mean, median, standard deviation, P95 latency, and FPS derived from mean latency. 5. Evaluate COCO8 precision, recall, mAP50, and mAP50–95. 6. Save environment details with the results.

Reproduce in Colab

```
%cd /content/Task2_CV_Optimization
!python src/benchmark.py --device 0 --runs 100 --warmup 20
```

The environment used for this run: Python 3.13.15, PyTorch 2.11.0+cu130, Ultralytics 8.4.182, OpenCV 4.14.0, CUDA 13.0, Tesla T4 GPU.

9. Recommendations for Future Work

1. Benchmark end-to-end video. Run the real webcam or a fixed video file through capture, preprocessing, detection, tracking, trajectory drawing, and display. Report both model latency and end-to-end FPS, and measure each stage separately.

2. Use a larger, fixed validation set. COCO8 is only a smoke test. Compare all configurations on the same larger dataset and, ideally, on data representative of the intended application. Include confidence intervals or multiple benchmark repetitions.

3. Profile before optimizing further. Use PyTorch Profiler or an equivalent profiler to identify CPU/GPU time, kernel activity, memory copies, post-processing, and tracker overhead. Do not assume inference is the dominant bottleneck without measurement.

4. Test ONNX and TensorRT. Export the same checkpoint to ONNX and, where supported, TensorRT. Benchmark each runtime using equivalent input shapes, warm-up, synchronization, and accuracy checks.

5. Evaluate INT8 quantization carefully. Use a supported export/runtime path and representative calibration data. Compare latency, model size, precision, recall, mAP, and any deployment-specific failure cases.

6. Investigate structured pruning only if justified. Pruning can reduce parameters or operations, but unstructured sparsity does not necessarily produce faster inference unless the runtime/hardware exploits it. Validate the accuracy impact after fine-tuning.

7. Improve trajectory handling. Keep a bounded history per track, avoid redrawing every historical segment if not needed, and profile the tracker and visualization separately. Verify that trajectory overlays are drawn on the exact frame displayed.

8. Report variability. Compare P95 latency as well as mean latency. Repeat measurements and keep the hardware, software, image size, model checkpoint, and timing boundaries fixed.

10. Conclusion

Three YOLOv8n configurations were evaluated using measured Colab results. The FP32 640 baseline achieved 7.547 ms mean latency and 132.51 derived FPS. FP16 at 640 and FP16 at 512 were slower in this experiment, so neither was demonstrated to be a latency optimization on the tested setup. Detection metrics varied across the configurations, but the COCO8 dataset is too small to support broad conclusions about accuracy preservation.

The appropriate engineering conclusion is to keep the measured baseline as the current reference, profile the complete application, and test alternative runtimes and precision formats under controlled conditions. Optimization should be accepted only when it produces a repeatable improvement while satisfying accuracy and application-level requirements.

Appendix A. Result Files

The submission bundle includes the benchmark source scripts, benchmark notebook, requirements file, raw benchmark CSV and JSON, environment metadata, measured-results Markdown, generated charts, and this report. Raw measurements are retained so the comparison can be independently reviewed.